

✓ 1. What is Testware?

Testware is test artifacts like test cases, test data, test plans needed to design and execute a test.

✓ 2. What is the difference between build and release?

Build: It is a number given to Installable software that is given to the testing team by the development team.

Release: It is a number given to Installable software that is handed over to the customer by the tester or developer.

✓ 3. What are the automation challenges that SQA(Software Quality Assurance) team faces while testing?

- ✓ Mastering the automation tool
- ✓ Reusability of Automation script
- ✓ Adaptability of test case for automation
- ✓ Automating complex test cases.

Bug Release is when a known defect is intentionally allowed in the release because it's deemed to have low impact, not urgent, or necessary to meet deadlines.

✓ 4. What is bug leakage and bug release?

Bug release is when software or an application is handed over to the testing team knowing that the defect is present in a release. During this the priority and severity of bug is low, as bug can be removed before the final handover.

Bug leakage is something, when the bug is discovered by the end users or customer, and not detected by the testing team while testing the software.

✓ 5. What is data driven testing?

Data driven testing is an automation testing ^{technique} framework, which tests the different input values on the AUT. These values are read directly from the data files. The data files may include csv files, excel files, data pools and many more.

used when you need to test the same functionality with different data inputs.

6. Explain the steps for Bug Cycle?

- ✓ Once the bug is identified by the tester, it is assigned to the development manager in open status
- ✓ If the bug is a valid defect the development team will fix it.
- ✓ If it is not a valid defect, the defect will be ignored and marked as rejected
- ✓ The next step will be to check whether it is in scope. If the bug is not the part of the current release then the defects are postponed
- ✓ If the defect or bug is raised earlier then the tester will assign a DUPLICATE status
- ✓ When bug is assigned to developer to fix, it will be given a IN-PROGRESS status
- ✓ Once the defect is repaired, the status will change to FIXED at the end the tester will give CLOSED status if it passes the final test.

7. What does the test strategy include?

The test strategy includes an introduction, resource, scope and schedule for test activities, test tools, test priorities, test planning and the types of test that has to be performed.

8. What is branch testing and what is boundary testing?

The testing of all the branches of the code, which is tested once, is known as branch testing. While the testing, that is focused on the limit conditions of the software is known as boundary testing.

9. What are the contents of test plans and test cases?

- ✓ Testing objectives
- ✓ Testing scope
- ✓ Testing the frame
- ✓ The environment
- ✓ Reason for testing
- ✓ The criteria for entrance and exit
- ✓ Deliverables
- ✓ Risk factors

10. What is the strategy for Automation Test Plan?

- ✓ The strategy for Automation Test Plan
- ✓ Preparation of Automation Test Plan
- ✓ Recording the scenario
- ✓ Error handler incorporation
- ✓ Script enhancement by inserting check points and looping constructs
- ✓ Debugging the script and fixing the issues
- ✓ Rerunning the script
- ✓ Reporting the result

11. What is quality audit?

The systematic and independent examination for determining the effectiveness of quality control procedures is known as the quality audit.

12. Explain stress testing, load testing and volume testing?

- ✓ Load Testing: Testing an application under heavy but expected load is known as Load Testing. Here, the load refers to the large volume of users, messages, requests, data, etc.
- ✓ Stress Testing: When the load placed on the system is raised or accelerated beyond the normal range then it is known as Stress Testing.
- ✓ Volume Testing: The process of checking the system, whether the system can handle the required amounts of data, user requests, etc. is known as Volume Testing.

13. What are the five common solutions for software developments problems?

- ✓ Setting up the requirements criteria, the requirements of a software should be complete, clear and agreed by all
- ✓ The next thing is the realistic schedule like time for planning , designing, testing, fixing bugs and re-testing
- ✓ Adequate testing, start the testing immediately after one or more modules development.
- ✓ Use rapid prototype during design phase so that it can be easy for customers to find what to expect
- ✓ Use of group communication tools [use case describes how a system interacts with users](#)

✓ 14. What is a 'USE' case and what does it include?

The document that describes, the user action and system response, for a particular functionality is known as USE case. It includes revision history, table of contents, flow of events, cover page, special requirements, pre-conditions and post-conditions.

15. What is CRUD testing and how to test CRUD?

CRUD stands for Create, Read, Update and Delete. CRUD testing can be done using SQL statements.

16.. What is thread testing?

A thread testing is a top-down testing, where the progressive integration of components follows the implementation of subsets of the requirements, as opposed to the integration of components by successively lower levels.

✓ 17. What is configuration management?

It is a process to control and document any changes made during the life of a project. Release control, Change control and Revision control are the important aspects of configuration management.

18. What is Ad Hoc testing?

It is a testing phase where the tester tries to break the system by randomly trying the system's functionality. It can include negative testing as well.

19. List out the roles of Software Quality Assurance engineer?

A software quality assurance engineer tasks may include following things amongst others

- ✓ Writing source code
- ✓ Software design
- ✓ Control of source code
- ✓ Reviewing code
- ✓ Change management
- ✓ Configuration management
- ✓ Integration of software
- ✓ Program testing
- ✓ Release management process

20. Explain what are test driver and test stub and why it is required?

- ✓ The stub is called from the software component to be tested. It is used in top down approach
- ✓ The driver calls a component to be tested. It is used in bottom up approach
- ✓ It is required when we need to test the interface between modules X and Y and we have developed only module X. So we cannot just test module X but if there is any dummy module we can use that dummy module to test module X

✓ 21. Explain what is Bug triage?

A bug triage is a process to

- ✓ Ensure bug report completeness
- ✓ Assign and analyze the bug
- ✓ Assigning bug to proper bug owner
- ✓ Adjust bug severity properly
- ✓ Set appropriate bug priority

bug triage is to ensure that bugs are addressed in an efficient and organized manner, based on their severity, impact,

22. List out various tools required to support testing during development of the application?

To support testing during development of application following tools can be used

- ✓ Test Management Tools: JIRA, Quality Center etc.
- ✓ Defect Management Tools: Test Director, Bugzilla
- ✓ Project Management Tools: Sharepoint
- ✓ Automation Tools: RFT, QTP, and WinRunner

✓ 23. What is a cause effect graph?

A cause effect graph is a graphical representation of inputs and the associated

visual tool that helps testers design test cases based on the logical relationships between inputs and outputs

outputs effects that
can be used to design test cases.

24. What is a Test Metric in software testing and what information does it contains?

In software testing, Test Metric is referred to the standard of test measurement. They are the statistics narrating the structure or content of a testing. It contains information like

- ✓ Total test
- ✓ Test run
- ✓ Test passed
- ✓ Test failed
- ✓ Tests deferred
- ✓ Test passed the first time

25. Explain what is traceability matrix?

A test matrix is used to map test scripts to requirements.

26. Explain what is the difference between Regression testing and Retesting?

Retesting is carried out to check the defects fixes, while regression testing is performed to check

whether the defect fix have any impact on other functionality.

27. List out the software quality practices through the software development cycle?

Software quality practices includes

- ✓ Review the requirements before starting the development phase
- ✓ Code Review
- ✓ Write comprehensive test cases
- ✓ Session based testing
- ✓ Risk based testing
- ✓ Prioritize bug based on usage
- ✓ Form a dedicated security and performance testing team
- ✓ Run a regression cycle
- ✓ Perform sanity tests on production
- ✓ Simulate customer accounts on production
- ✓ Include software QA Test Reports

✓ 28. Explain what is the rule of a "Test Driven Development"?

The rule of a Test Driven Development is to prepare test cases before writing the actual code.

Which means you are actually be writing code for the tests before you write code for the application.

29. Mention what are the types of documents in SQA?

The types of documents in SQA are

- ✓ Requirement Document
- ✓ Test Metrics
- ✓ Test cases and Test plan
- ✓ Task distribution flow chart
- ✓ Transaction Mix
- ✓ User profiles
- ✓ Test log
- ✓ Test incident report
- ✓ Test summary report

30. Explain what should your QA documents include?

QA testing document should include

- ✓ List the number of defects detected as per severity level
- ✓ Explain each requirement or business function in detail
- ✓ Inspection reports

- ✓ Configurations
- ✓ Test plans and test cases
- ✓ Bug reports
- ✓ User manuals
- ✓ Prepare separate reports for managers and users

31. Explain what is MR and what information does MR consists of?

MR stands for Modification Request also referred as Defect report. It is written for reporting errors/problems/suggestions in the software.

32. Mention how validation activities should be conducted?

Validation activities should be conducted by following techniques

- ✓ Hire third party independent verification and validation
- ✓ Assign internal staff members that are not involved in validation and verification activities
- ✓ Independent evaluation

33. What do you understand about Traceability Matrix (TM) in the context of quality assurance?

A Traceability Matrix is a document that connects any two baseline documents that require a

many-to-many link to ensure that the relationship is complete. It's used to keep track of

requirements and make sure they're being met on the present project.

The following are the three major components of a traceability matrix:

- ✓ Forward Traceability: This matrix is used to determine whether the project is progressing in the appropriate direction and for the correct product. It ensures that each specification is applied to the product and that each specification is adequately tested. It connects test cases to requirements.

- ✓ Backward or reverse Traceability: It is used to verify that the present product is still on track.

The goal of this form of traceability is to ensure that we are not expanding the project's scope by

adding code, design features, testing, or other activities not stated in the requirements. It connects requirements to test cases.

- ✓ Bidirectional Traceability: This traceability matrix ensures that all criteria are covered by test cases in both directions (forward and backward). It examines the impact of a change in requirements caused by a work product defect, and vice versa.

34. What do you understand about defect leakage ratio in the context of quality assurance?

Software testers utilise defect leakage as a metric to determine the effectiveness of Quality

Assurance (QA) testing. It's the ratio of the total number of flaws attributed to a stage (which are

captured in subsequent stages) to the sum of the total number of defects captured in that stage and

the total number of defects assigned to a stage (which are captured in subsequent stages). Defect

leakage is a metric that counts the percentage of faults that leak from one testing stage to the next,

as well as demonstrating the effectiveness of software testers' testing. The testing team's worth, on

the other hand, is only confirmed when defect leaking is small or non-existent.

35. What do you mean by monkey testing in the context of quality assurance?

Monkey testing is a software testing technique in which the tester inserts any

random inputs into the software application without using predefined test cases and observes the software program's behaviour to see if it crashes. The goal of monkey testing is to use experimental ways to uncover faults and problems in software applications. Monkey testing is a sort of black-box testing that involves supplying random inputs to a system in order to check its behaviour, such as whether it is crashing or not. Monkey testing does not necessitate the creation of test cases. It can also be automated, in the sense that we can develop programs or scripts to produce random inputs in order to test the system's behaviour. When undertaking stress or load testing, this technique comes in handy.

Monkeys are divided into two categories:

- ✓ Smart Monkeys: The smart monkeys are those who have a basic understanding of the application. They know which of an application's pages will redirect to which page. They also know whether the inputs they're giving are valid or not. If they discover an error, they are wise enough to report it as a bug. They are also aware of the menus and buttons.
- ✓ Dumb Monkeys: Dumb Monkeys are individuals who are completely unaware of the application. They have no idea where an application's pages will reroute. They give random inputs and are unaware of the application's beginning and finish points. Despite the fact that they have no knowledge of the application, they discover bugs such as environmental failure or hardware failure. They also have limited knowledge of an application's functioning and user interface

36. What do you mean by gorilla testing in the context of quality assurance?

Gorilla Testing:

Gorilla testing is a method of software testing in which a module is frequently tested based on some random inputs, ensuring that the module's operations are checked and that there are no problems in the module. So it's also known as Torture Testing, Fault Tolerance Testing, or Frustrating Testing because of the Gorilla Testing pattern. It's a manual test that's done over and over again. In Gorilla Testing, testers and developers work together to regularly evaluate a module's functionality.

37. What are the various artifacts to which you refer when writing test cases?

Following are the various artifacts that we can refer to while writing test cases :

- ✓ Specification of functional requirements.
- ✓ Document that explains what the requirements are.
- ✓ Wireframes.
- Use Cases.
- ✓ User Stories.
- ✓ Acceptance criteria.
- ✓ User Acceptance Testing Test cases.

38. What do you understand about regression testing? Which test cases should be selected for regression testing?

Regression Testing is a sort of software testing used to ensure that a recent

program or code modification hasn't broken existing functionality. Regression Testing is just a full or partial re execution of previously executed test cases to confirm that current functionality is working properly. This testing ensures that new code modifications do not have unintended consequences for current functionality. It ensures that the old code continues to function after the most recent code modifications have been made.

According to industry data, a large proportion of defects reported by customers were caused by last-minute bug patches that had unintended consequences, making selecting the Test Case for regression testing an art and not an easy task. The following test scenarios can be used to create effective regression tests :

- ✓ Test scenarios with a high number of flaws
- ✓ Test cases covering features that are more visible to the users
- ✓ Test cases that test the product's fundamental features
- ✓ Test cases of functionalities that have experienced significant and recent alterations
- ✓ All Test Cases for Integration
- ✓ All Complex Test Cases
- ✓ Cases of boundary value tests.

39. Explain risk in the context of quality assurance. What are the five dimensions of risk?

When it comes to software testing, Risks are potential issues that could jeopardise the project stakeholders' goals. It's the likelihood of a bad or unfavourable result. A risk is something that has not yet occurred and may never occur; it is a potential concern. The likelihood of a risk becoming an outcome is proportional to the degree of risk linked with the potential for negative consequences.

Most people, for example, are expected to have a cold at some point in their lives, frequently multiple times. A healthy person, on the other hand, is spared any major effects. As a result, this individual's overall risk of catching a cold is low. On the other hand, for an elderly person with breathing issues, the risk of catching a cold is significant. As a result, the overall chance of catching a cold in his instance is significant.

Following are the five dimensions of risks:

- ✓ Schedule: Unrealistic schedules, the omission of important activities when drafting a schedule, and other factors could stymie project completion on time. If testing is done from a remote place, an unstable communication link can be regarded as a possible danger.
- ✓ Client: Ambiguous requirements description, lack of clarity on issues, frequent changes to requirements, and other factors could lead to chaos throughout project execution.
- ✓ Human Resources: Insufficient resources with the required skill level are not available for the project; Resource attrition - Appropriate training schedules for resources must be designed to balance the knowledge level with resources quitting. Underestimating the training effort could have a negative influence on project completion.

- ✓ System Resources: The inability to obtain or the delay in obtaining all key computer resources, such as hardware and software tools, or software licences, will have a negative impact.
- ✓ Quality: A combination of issues such as a shortage of resources, a tight delivery timetable, and frequent changes to requirements will have an impact on the product tested quality

40. What do you mean by quality audit in the context of quality assurance?

An audit is an on-site verification activity of a processor quality system, such as inspection or examination. An internal or external quality auditor, or an audit team, conducts a systematic analysis of a quality system as part of a quality audit. Quality audits are conducted at predetermined intervals to ensure that the institution's internal system monitoring methods are well defined and linked to successful action. Audits are an important management tool for confirming objective proof of processes. In other words, a quality audit is a verification effort aimed at determining the degree of compliance of a product, design, process, or system to a standard specification or procedure. The quality audit includes two parts: the first is an examination of the system in which the products or services are created, known as the quality system audit. The other is a product or service quality audit, which is an examination of the products themselves.

41. How do you figure out how much testing each piece of software requires in the context of Quality Assurance?

The Cyclomatic Complexity can be used to determine how much testing each piece of software requires in our application. The technique aids in the identification of the three questions listed below for the programs/features.

- ✓ Is it possible to test the feature or programme?
- ✓ Is everyone aware of the feature/program?
- ✓ Is the feature/program trustworthy?

We can use this technique to determine the "level" of testing required for our application. It is standard practice to consider a piece of functionality to be complex if the cyclomatic complexity result is greater than or equal to a certain number, and to conclude as a tester that the piece of code/functionality requires in-depth testing. If the Cyclomatic Complexity result is a lesser value, we conclude as QA that the functionality is of lower complexity and adjust the scope accordingly. Understanding the full testing life cycle is critical, and we should be prepared to suggest modifications to our approach if necessary. The goal is to deliver high-quality software, so a QA should take all necessary steps to improve the testing process and the manner the testing team conducts the tests.

42. What do you expect out of this role?

This question measures your compatibility with the role. As you have applied for the role, you should have a good understanding of the major responsibilities associated with it and what you

expect from it in the future.

43. Did you handle a team of test engineers or QA professionals in your earlier role?

As you are interviewing for a test lead position it is evident that you have some amount of experience in handling a team or have similar experiences. You need to give a detail on how you coordinated with your team members and clients to deliver successful projects.

44. What do you think of test leads writing test cases?

The right answer is to say that you feel that the test lead should write test cases like any other member of the team.

45. How do you set your team's objectives?

If you prefer to set individual objectives for each member of the team, mention that you set them according to the knowledge and experience levels. This is how we can handle the project more efficiently as a team.

46. How familiar are you with the industry-standard testing practices?

Explain all the proper practices that you follow in your existing organization and how will you handle it similarly for the organization that you are interviewing for.

47. What are the ways you ensure that the team members receive proper training?

Get feedback from all the team members on their strengths and weaknesses to make a note of what type of training is necessary for the team. Also, new members who join the team should be trained on time so as to be inducted as quickly as possible.

48. What will be your criteria for hiring team members?

The criteria for hiring team members are 1) his/her technical strength is as per project requirements, 2) his/her attitude towards the profile he will be hiring for, and 3) will he/she be a good fit with the rest of the team members?

49. Which testing tools are you familiar with?

This is one of the most important Test lead interview questions. As a test lead, you should be familiar with some of the most popular testing tools. You need to explain how you use it and what are its advantages as per the project requirements, cost of using the tool, and ease of use.

50. Are you familiar with automation and what do you think of it?

Automation is becoming an important part of software testing nowadays and you can get away without having any knowledge of it. You should be well-prepared for this Test Lead interview question and make it a positive answer as automation is going to be the future.

51. Name the different types of Test Plans?

There are three types of Test Plans: 1) Master Test Plan, 2) Testing Level Specific Test Plan and 3) Testing Type Specific Test Plan.

52. What are the risks that should be avoided for a testing project?

One should avoid the following risks during a testing project: 1) human resource risk (resource

crunch), 2) project schedule risk (missed deadlines), 3) strategy risk (exceeding allocated budgets) and 4) project definition risk.

53. What is Automation?

Automation is any action that can reduce human efforts.

54. What is Automation testing?

The process of using special software tools or scripts to perform testing tasks such as entering data, executing the test steps and comparing the results, etc. is known as Automation testing.

55. When is Automation testing useful?

Automation testing is useful in the following scenarios:

- a) Regression testing: In case of a bug fix or new module implementation, we have to make sure that the already implemented or unchanged functionality is not affected. In this case, we end up running the regression test case multiple times.
For Example: After each change request or bug fix, after each iteration in case of incremental development approach, etc.
- b) Non-functional Testing: Testing the non-functional aspects of an application. For Example, Load testing or performance testing, etc are very difficult for humans to track and analyze.
- c) Complex calculation checks or test scenarios that are prone to human errors.
- d) Repeated execution of the same tests: Sometimes, we have to run the same set of test case for a different set of data or after each build release or on multiple hardware, software or combination of both.

Automating the test cases in the above scenarios helps in achieving the speed of testing and minimizing human errors.

56. Can you achieve 100% automation?

100% automation would be difficult to achieve because there would be many edge test cases and some cases that are executed seldom. Automating these cases which are not executed that often will not add value to the automated suite.

57. How to decide the tool that one should use for Automation testing in their projects?

In order to identify the tool for Automation testing in your project:

- a) Understand your project requirements thoroughly and identify the testing scenarios that you want to automate.
- b) Search for the list of tools that support your project's requirements.
- c) Identify your budget for the automation tool. Select the tools within your budget.
- d) Identify if you already have skilled resources for the tools. If you don't have the necessary skilled resources then identify the cost for training the existing resources or hiring new resources.
- e) Now compare each tool for key criteria like:
 - ✓ How easy is it to develop and maintain the scripts for the tool?
 - ✓ Can a non-technical person also execute the test cases with little training?
 - ✓ Does the tool support different types of platforms like web, mobile, desktop, etc based on your project requirements?

- ✓ Does the tool have a test reporting functionality? If not, is it easily configurable for the tool?

- ✓ How is the tool for cross-browser support for web-based applications?
- ✓ How many different testing types can this tool support?
- ✓ How many languages does the tool support?

f) Once you have compared the tools, select the tool which is within your budget and support your project requirements, and gives you more advantages based on the key criteria mentioned above.

58. Currently I do not have any automation in place in my project, but now I want to implement automation, what would be my steps?

- ✓ First, identify which type of testing/test cases you want to automate.
- ✓ Identify the tool
- ✓ Design the framework
- ✓ Create utility files and environment files.
- ✓ Start scripting
- ✓ Identify and work on reporting.
- ✓ Allocating time for enhancing and maintaining the scripts.

Steps required for getting Automation Testing in place for a project include:

- ✓ Understand the advantages and disadvantages of automation testing and identify the test scenarios which are suitable for automation.
- ✓ Select the automation tool that is best suited for automating the identified scenarios
- ✓ Find the tool expert to help in setting up the tool and required environment for executing the test cases using the tool.
- ✓ Train the team so that they can write scripts in the programming language that the tool supports.
- ✓ Create the test framework or identify the already existing one that meets your requirements.
- ✓ Write an execution plan for OS, browsers, mobile devices, etc.
- ✓ Write programming scripts for manual test cases to convert them into automated test cases.
- ✓ Report the test case status by using the reporting feature of the tool.
- ✓ Maintain the scripts for ongoing changes or new features.

59. What is a framework?

A framework is a set of the structure of the entire automation suite. It is also a guideline, which if followed can result in a structure that is easy to maintain and enhance. These guidelines include:

- ✓ Coding standards
- ✓ Handling the test data
- ✓ Maintaining and handling the elements (object repository in QTP)
- ✓ Handling of environment files and properties file
- ✓ Reporting of data
- ✓ Handling logs

60. What are the attributes of a good framework?

The characteristics include:

- ✓ Modular: The framework should be adaptable to change. Testers should be able to modify the scripts as per the environment or login information change.
- ✓ Reusable: The commonly used methods or utilities should be written in a common file that is accessible to all the scripts.
- ✓ Consistent: The suite should be written in a consistent format by following all the accepted coding practices.

- ↳ Independent: The scripts should be written in such a way that they are independent of each other. In case one test fails, it should not hold back the remaining test cases (unless it is a login page)
- ↳ Logger: It is good to have implemented the logging feature in the framework. This would help in case our scripts run for longer hours (say nightly mode), if the script fails at any point of time, having the log file will help us to detect the location along with the type of the error.
- ↳ Reporting: It is good to have the reporting feature automatically embedded into the framework. Once the scripting is done, we can have the results and reports sent via email.
- ↳ Integration: Automation Framework should be such that it is easy to integrate with other applications like continuous integration or triggering the automated script as soon as the build is deployed.

61.Can you do without a framework?

Frameworks are guidelines and not mandatory rules, so we can do without a framework, but if we create it and follow it, enhancing and maintaining would be easy to implement.

62.What are the different types of the Automation tool that you are aware of?

Open source tool like Selenium, JMeter, etc.

Paid tools like QTP, Load Runner, Ranorex, RFT, and Rational Robot.

63. What generally is the structure of a framework?

Normally the structure should have - (It would differ from project to project)

- ↳ A "src" (source) folder having the actual test scripts.
- ↳ A "lib" (library) folder having all the libraries and common methods.
- ↳ A "class" folder having all the class file (in-case using java).
- ↳ A "log" folder having the log file(s).
- ↳ A file/folder having all the web element Ids.
- ↳ A file containing the URL, environment and login information.

64. Where will you maintain information like URL, login, password?

This information should always be maintained in a separate file.

65. Why do you want to keep this kind of information in a separate file and not directly in the code?

URL, Login, and passwords are the kind of fields that are used very often and these change as per the environment and authorization. In case we hardcode it into our code, we have to change it in every file which has its reference.

In case if there are more than 100 files, then it becomes very difficult to change all the 100 files

and this, in turn, can lead to errors. So this kind of information is maintained in a separate file so that updating becomes easy.

66.What are the different types of frameworks?

Different types of frameworks includes:

- ↳ Keyword-driven framework
- ↳ Data-Driven framework
- ↳ Hybrid Framework
- ↳ Linear Scripting

67.Can you tell some good coding practices while automation?

Some of the good coding practices include:

- ✓ Add appropriate comments.
- ✓ Identify the reusable methods and write it in a separate file.
- ✓ Follow the language-specific coding conventions.
- ✓ Maintain the test data in a separate file.
- ✓ Run your scripts regularly.

68.Any kind of test which you think should not be automated?

- ✓ Tests that are seldom executed.
- ✓ Exploratory testing
- ✓ Usability testing
- ✓ Test which is executed quickly when done manually.

69.Do you think that testing can be done only at the UI level?

Today as we are moving to the Agile mode, testing is not limited to the UI layer. Early feedback

is imperial for an agile project. If we concentrate only on the UI layer, we are actually waiting until the UI is developed and available to test.

Rather we can test even before the UI is actually developed. We can directly test the APIs or the methods using tools like Cucumber and FitNesse.

In this way, we are giving the feedback much early and are testing even before the UI is developed.

Following this approach will help us to test only the GUI aspect of small cosmetic changes or some validations on the UI and will help the developers by giving more time to fix the bugs.

70. How do you select which automation tool is best suited for you?

Selecting the automation tool depends upon various factors like:

- ✓ The scope of the application which we want to automate.
- ✓ Management overhead like cost and budget.
- ✓ Time to learn and implement the tool.
- ✓ Type of support available for the tool.
- ✓ Limitation of the tool

71.What do you think holds the testers back to do automation? Is there a way to overcome it?

The major hurdle for testers is to learn programming/coding when they want to automate. Since

testers do not code, adapting to coding is a bit challenging for testers.

We can overcome it by:

- ✓ Collaborating with developers when automating.
- ✓ Considering that automation is the responsibility of the whole team and not only of the testers.
- ✓ Giving a dedicated time and focus on automation.
- ✓ Getting proper management support.

72.What are the important modules of an automation testing framework?

Important modules of an Automation testing framework are:

1. Test Assertion Tool: This tool will provide assert statements for testing the expected

values in the application under test. For Example. TestNG, Junit, etc.

2. Data Setup: Each test case needs to take the user data either from the database or from a

file or embedded in the test script. Frameworks data module should take care of the data

intake for test scripts and the global variables.

3. Build Management Tool: Framework needs to be built and deployed for the use of

creating test scripts.

4. Continuous integration tool: With CICD (Continuous Integration and Continuous Development) in place, continuous integration tool is required for integrating and deploying the changes done in the framework at each iteration.

5. Reporting tool: A reporting tool is required to generate a readable report after the test

cases are executed for a better view of the steps, results, and failures.

6. Logging tool: The logging tool in framework helps in better debugging of the error and bugs.

73. What are the different types of testing framework techniques?

There are four types of automation testing framework techniques.

They are:

(i) Modular Testing framework:

This framework is built on the concept of abstraction. In this framework, the tester creates scripts

for each module of the application under test individually and then these scripts are combined in

the hierarchical order to create large test cases.

It creates an abstraction layer between the modules, thus any modifications in test scripts for one

module do not affect any other modules.

Advantages of this framework:

- ✓ Easier maintenance and scalability of test cases.

- ✓ Creating test cases by using already scripted modules is easier and faster.

Disadvantages:

- ✓ Test cases have data embedded in them. Thus executing the same test script with different

data is a big change at the script level.

(ii) Data-Driven Testing framework:

In the Data-driven testing framework, the input data and the expected output data corresponding

to the input data is stored in a file or database and the automated script runs the same set of test

steps for multiple sets of data. With this framework, we can run multiple test cases where only the

input data differs and the steps of execution are the same.

Advantages:

- ✓ Reduces the number of test scripts that are required to be executed. We execute the same

script multiple times with different data.

- ✓ Less coding for automation testing.

- ✓ Greater flexibility for maintaining and fixing the bugs or enhancing the functionality.

- ✓ Test data can be created even before the automated system for testing is ready.

Disadvantages:

- ✓ Only similar test cases with the same set of execution steps can be combined for multiple

sets of data. The different set of execution steps require a different test case.

(iii) Keyword-Driven Testing framework:

It is an application-independent testing framework that uses data tables and self-explanatory

keywords. Keywords explain the actions to be performed on the application under test and the data

table provides the input and expected output data.

Keyword-based testing is an increment to data-driven testing.

Advantages:

- ✓ Less coding and the same script can be used for multiple sets of data.
- ✓ Automation expertise is not required for creating a test case using the already existing keywords for actions.
- ✓ Same keywords can be used across multiple test cases.

Disadvantages:

- ✓ This framework is more complicated as it needs to take care of the keyword actions and also the data input.
- ✓ Test cases get longer and complex thereby affecting the maintainability of the same.

(iv) Hybrid Testing framework:

This framework is a combination of all the above-mentioned testing frameworks (Modular, Data driven, and Keyword-driven).

In this framework, the test cases are developed from modular scripts by combining them in the modular testing framework. Each of the test cases uses a driver script that uses a data file as in the data-driven framework and a keyword-based action file.

Advantages:

- ✓ Modular and easy to maintain.
- ✓ Less coding can take care of more test cases.
- ✓ One test case can be executed with multiple sets of data.

Disadvantages:

- ✓ Complex to read, maintain and enhance.

74. When do you prefer Manual testing over Automation testing?

Answer: We prefer manual testing over automation testing in the following cases:

- ✓ The project is short-term and writing scripts will be time-consuming and costly when compared to manual testing.

Flexibility is required. Automated test cases are programmed and run in a specific way of configurations.

- ✓ Usability testing needs to be performed.
- ✓ Applications/module is newly developed and has no previous test cases.
- ✓ Ad-hoc or exploratory testing needs to be performed.

75. How will you automate basic "login" functionality test cases for an application?

Assuming that the automation tool and framework is already in place of the test environment.

To test the basic "Login" functionality:

- ✓ Understand the project requirement: Login functionality will have a username textbox, a password textbox, and a login button.

Identify the Test scenarios: For login functionality, the possible test scenarios are:

- ✓ Blank username and password
- ✓ Invalid username and password
- ✓ A valid username and invalid password
- ✓ Valid username and password
- ✓ Prepare a Data input file with the data corresponding to each scenario.
- ✓ Launch the tool from the program.
- ✓ Identify the username field, password field, and the login button.
- ✓ For each test scenario, get the data from the data file and enter into the corresponding fields.

Program click on the login button after entering the data.

- ✓ Validate the error message for negative scenarios and the success message for positive

scenarios in the test script with the help of assertions.

- ✓ Run the test suite and generate the report.

76. Is Automation testing a Black box testing or White-box testing?

Automation testing is mostly a black box testing as we just program the steps that a manual tester performs for application under test without knowing the low-level design or code of the application. Sometimes, automated test scripts need access to the database details that are used in the application under test or some more coding details and thus can be a type of white-box testing. Thus automated testing can be both black or white box type of testing depending on the scenarios in which automation is performed.

77. How many test cases have you automated per day?

Well, the number depends on the complexity of the test cases. When the complexity was limited, I was able to automate 5 to 6 test cases per day. Sometimes, I was able to automate only one test case for complex scenarios. I have also broken down my test cases into different components like, take input, do the calculation, verify the output etc. in case of very complex scenarios and have taken 2 or more days.

78. What factors determine the effectiveness of Automation testing?

Answer: Some of the factors that determine the effectiveness of automation testing are:

- ✓ Time saved by running scripts over the manual execution of test cases.
- ✓ Defects found
- ✓ Test Coverage or code coverage
- ✓ Maintenance time or development time
- ✓ Stability of the scripts
- ✓ Test Reusability
- ✓ Quality of the software under test

79. What is the scripting standard while performing automation testing?

Uniformity in naming convention, commenting on the functionality whenever and wherever necessary, adequate indentation, robust error handling, and the ability to recover easily are some good practices you should follow while scripting.

80. What are the important modules of the Test Automation Framework?

Test Assertion Tool: This module will provide assert statements for testing the expected values in the application under test.

Data Setup: Each test case needs to take the user data from the database, a file, or embedded in the test script.

Build Management Tool: Tool that you can use to build a framework so that you can create test scripts.

Continuous integration tool: Continuous Integration/Continuous Delivery tools are required for

integrating and deploying the changes done in the framework at each iteration.

Reporting tool: A reporting tool is required to generate a readable report after the test cases are

executed. This way, you can better view the steps, results, and failures.

Logging tool: The tool in the framework helps better debug errors and bugs.

81.What are Test Automation Framework development challenges?

Some of the challenges that you come across while using frameworks are:

- ✓ Ability to understand what needs to be achieved out of test automation
- ✓ Identifying requirements from multiple areas to design automation Framework
- ✓ Tool identification

82.What is data-driven testing?

The aim of data-driven automation testing is to simplify the testing process involving complex and

huge data sets. In Data Driven Testing, the test data includes input, expected output, and a result

field. These fields are listed in CSV files, excel files, text files, XML files, etc. These files are then

fed to the automation tool for execution, which compares the expected and actual data. Then the

obtained results are documented in the result field.

✓ 83. What is TestNG? List out some of its prominent features.

TestNG is an open-source automated testing framework, where NG refers to NextGeneration.The

creator of TestNG is Cedric Beust. It is inspired by other frameworks like JUnit and NUnit. It is

designed to be better than JUnit, especially while testing integrated classes.TestNG allows the

developer to write more flexible and powerful tests with ease. Some of its prominent features

include:

- ✓ Supports annotations
- ✓ Based on more Java and OO features
- ✓ Supports testing integrated classes
- ✓ Provides flexible runtime configuration
- ✓ Supports dependent test methods, parallel testing, and load testing
- ✓ Offers flexible plug-in API.
- ✓ And supports multithreading as well.

✓ 84.What are the advantages of TestNG over JUnit?

In TestNG, testing is based on JUnit, but it is designed to overcome the limitations of JUnit. Some

advantages of TestNG over JUnit are:

- ✓ Annotations are easier to understand in TestNG
- ✓ In TestNG, test cases can be grouped easily
- ✓ TestNG supports parallel testing, unlike in JUnit

✓ 85.What is TestNG? List out some of its prominent features.

TestNG is an open-source automated testing framework, where NG refers to NextGeneration.The

creator of TestNG is Cedric Beust. It is inspired by other frameworks like JUnit and NUnit. It is

designed to be better than JUnit, especially while testing integrated classes.TestNG allows the

developer to write more flexible and powerful tests with ease. Some of its prominent features

include:

- ✓ Supports annotations
- ✓ Based on more Java and OO features
- ✓ Supports testing integrated classes
- ✓ Provides flexible runtime configuration
- ✓ Supports dependent test methods, parallel testing, and load testing
- ✓ Offers flexible plug-in API.
- ✓ And supports multithreading as well.

